

CHAPTER 1

INTRODUCTION

Artificial intelligence has made remarkable strides in recent years, enabling transformative applications in healthcare, medical imaging, and diagnostics. Brain imaging techniques such as MRI scans provide crucial insights into neurodegenerative diseases like Alzheimer's. In traditional diagnostic methods rely on manual assessment by radiologists, which can be time-consuming and prone to subjective interpretation. Machine learning models, particularly Convolutional Neural Networks, have emerged as a powerful tool for automating the analysis of medical images, offering improved accuracy and efficiency in disease detection. By learning spatial hierarchies of features, CNNs can identify patterns within MRI scans that indicate different stages of Alzheimer's, paving the way for early diagnosis and intervention.

To further enhance classification accuracy, hybrid approaches integrating CNNs with SVMs have been explored. While CNNs excel at feature extraction, SVMs are effective at handling high-dimensional data and defining optimal decision boundaries. The CNN model extracts deep features from MRI images, which are subsequently fed into an SVM classifier for final prediction. Additionally, Grad-CAM is used to generate heatmaps, highlighting the regions in an MRI scan that contributed most to the model's decision. The explainability feature enhances trust in AI-driven diagnostics, ensuring that medical professionals can interpret and validate model predictions.

1.1 IMPORTANCE OF EARLY DETECTION IN BLOOD GROUP

Early detection of Alzheimer's is crucial for implementing timely interventions, slowing disease progression, and improving patient quality of life. Identifying cognitive decline at an early stage allows for medical treatments, lifestyle adjustments, and support systems to be introduced before irreversible damage occurs. Machine learning-based approaches significantly aid in detecting subtle structural changes in the brain that may not be immediately evident through traditional methods.

1.2 OVERVIEW OF MACHINE LEARNING IN MEDICAL FIELDS

Machine learning has revolutionized medical diagnostics by enabling automated and efficient disease detection, diagnosis, and treatment planning. Traditional diagnostic methods require manual analysis, which is time-consuming and prone to errors. Deep learning techniques like Convolutional Neural Networks (CNNs) enhance medical imaging analysis, enabling early detection of diseases such as Alzheimer's, cancer, and cardiovascular conditions. By extracting essential features from MRI, CT, and X-ray scans, these models improve diagnostic accuracy and support healthcare decision-making.

Beyond imaging, ML facilitates personalized treatment planning, risk prediction, and real-time patient monitoring. Combining SVMs with CNNs enhances classification accuracy, particularly in Alzheimer's detection, where CNNs extract brain scan features and SVMs refine classification. Explainability tools like Grad-CAM generate visual heatmaps, ensuring transparency in AI-driven diagnosis. Continuous learning capabilities enable ML models to adapt to new medical data, making healthcare more efficient, accessible, and data-drive.

1.3 AIM OF THE STUDY

Developing an AI-powered system for early Alzheimer’s detection using MRI scans to enhance diagnostic accuracy and efficiency. A hybrid approach combining Convolutional Neural Networks (CNNs) for feature extraction and Support Vector Machines (SVMs) for classification ensures reliable disease stage identification.

Incorporating Grad-CAM for explainability improves transparency in AI predictions. A user-friendly Streamlit interface enables easy MRI scan analysis, while real-time model retraining ensures adaptability to new data. The Model aims to provide a scalable, automated, and accessible tool for Alzheimer’s diagnosis.

1.4 MACHINE LEARNING

Machine Learning (ML) is a subset of artificial intelligence (AI) that focuses on developing algorithms and models that enable computers to learn from data. The primary goal is to allow computers to make predictions, decisions, or identifications without being explicitly programmed for a particular task.

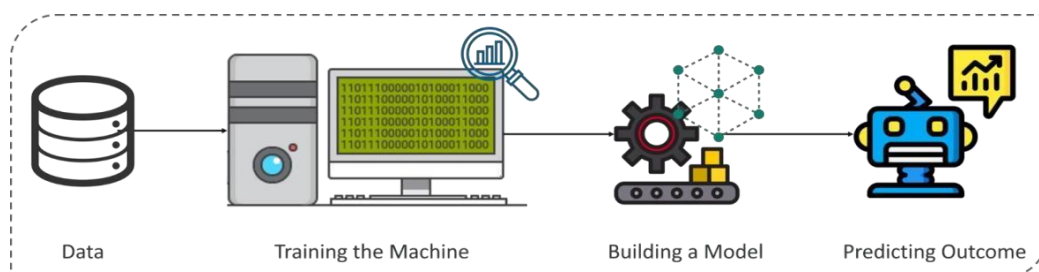


Fig 1.1 Concept of ML

1.5 TYPES OF MACHINE LEARNING

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning

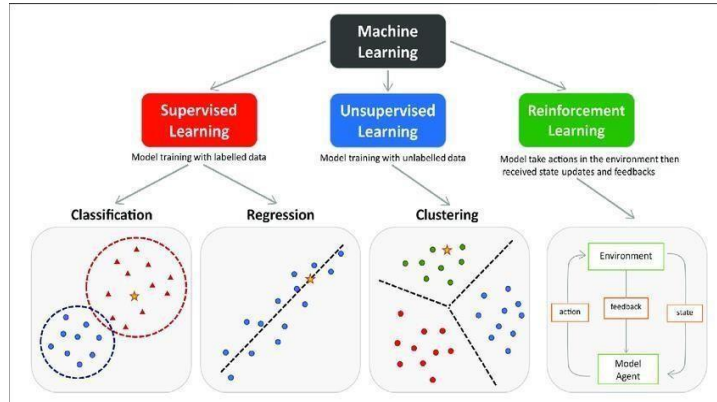


Fig 1.2 Types of Machine Learning

1.6 WORKING OF ML

Machine learning algorithms are molded on a training dataset to create a model. As new input data is introduced to the trained ML algorithm, it uses the developed model to make a prediction.

HOW DOES MACHINE LEARNING WORK?

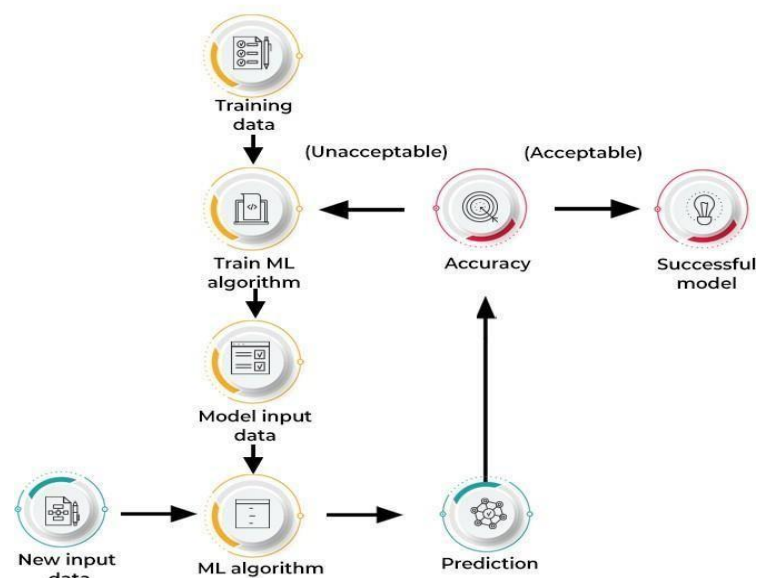


Fig1.3 Working of ML

1.7 RESEARCH MOTIVATION AND SIGNIFICANCE

Alzheimer's disease affects millions worldwide, and early detection is critical for effective treatment. Traditional diagnosis relies on manual MRI analysis, which is time-consuming and prone to errors. The Model aims to develop an AI-driven system using Convolutional Neural Networks (CNNs) for feature extraction and Support Vector Machines (SVMs) for classification, improving accuracy and efficiency.

The significance of this research lies in its use of Grad-CAM for explainability and a Streamlit-based interface for user accessibility.

1.8 ADVANTAGES

- Combines CNN for feature extraction and SVM for classification, offering good performance for image-based tasks.
- Modular structure makes the code easy to maintain and extend.
- Includes Grad-CAM for model explainability.
- Provides real-time model retraining, which improves accuracy overtime.
- Stores prediction history in a database for future reference.
- Uses Streamlit for an intuitive, user-friendly interface.
- Preprocessing and data normalization are well-handled to ensure consistent input to the models.

1.9 DISADVANTAGES

- Hardcoded file paths could cause issues if the project is moved to a different system.
- The CNN model is basic and lacks advanced techniques like dropout or data augmentation.

- Potential overfitting with small datasets, as no regularization is used.
- Limited performance on CPU; GPU acceleration would improve efficiency.
- No automated hyperparameter tuning for model optimization.
- The database only stores predictions, missing useful model or training data details.
- Dependencies on external libraries could cause compatibility issues during deployment.
- Lacks user authentication, which might be necessary for sensitive data handling.

1.10 RESEARCH OBJECTIVES

To develop an automated system for early-stage Alzheimer's disease detection using MRI scans by leveraging Convolutional Neural Networks for feature extraction and Support Vector Machines for classification, integrated with Grad-CAM for explainability, and a real-time retraining mechanism to enhance model performance over time. The system also includes a user-friendly Streamlit interface for predictions, prediction history storage, and model management.

CHAPTER 2

SURVEY ON LEVERAGING CNN AND SVM FOR EFFECTIVE ALZHEIMER'S DISEASE CLASSIFICATION

A Review on Alzheimer's Disease Through Analysis of MRI Images Using Deep Learning Techniques

Battula Srinivasa Rao and Mudiyaala Aparna(2023) present a comprehensive review of Alzheimer's disease detection using MRI image analysis with deep learning techniques. The study highlights the importance of early diagnosis through MRI-based brain structure segmentation, emphasizing the role of CNNs in classification. Various deep learning approaches, including single-modality and multi-modality segmentation techniques, are discussed to improve accuracy. The review also explores the challenges of MRI segmentation, dataset availability, and the necessity of automation for reliable AD classification. By integrating deep learning for MRI analysis, this method enhances diagnostic precision and aids in the early detection of Alzheimer's disease.

Applying Convolutional Neural Networks for Pre-detection of Alzheimer's Disease from Structural MRI Data.

Gunawardena et al. (2017) propose an automated system for the early detection of AD using deep learning techniques. The study explores image processing and CNNs to analyze MRI scans and identify AD symptoms

before irreversible brain damage occurs. The system involves preprocessing MRI images, applying segmentation techniques, and classifying brain structures using CNNs. The model demonstrated a high accuracy of 96%, outperforming traditional methods like SVMs. This approach provides a reliable, automated solution for early AD diagnosis, reducing dependency on manual evaluation.

Automatic Diagnosis of Alzheimer's Disease and Mild Cognitive Impairment Using CNN-SVM Networks.

Sun et al. (2021) propose a hybrid deep learning approach combining CNNs and SVMs for the automatic diagnosis of AD and MCI. The study utilizes 3D FDG-PET images and an end-to-end training strategy to classify patients into three categories: AD, MCI, and Normal Controls. The method leverages CNN's feature extraction capabilities and SVM's classification strength, achieving improved diagnostic accuracy compared to traditional CNN-based models. Experimental results show that the CNN-SVM framework enhances classification performance, offering a reliable tool for early AD prediction and aiding clinical decision-making.

Multi-Class Diagnosis of Alzheimer's Disease Using Hybrid CNN-SVM Model.

Ali Cherry et al. (2023) propose a hybrid CNN-SVM model for the early and accurate diagnosis of Alzheimer's disease using MRI images. The system processes a dataset of 3,000 images, employing preprocessing techniques such as resizing and normalization to enhance feature extraction. The CNN component extracts features, while the SVM classifier categorizes patients into non-demented, very mild demented, and demented classes. The

model achieves an impressive accuracy of 99.83%, with a sensitivity of 99.48% and specificity of 100%. The approach demonstrates significant potential in improving early detection and diagnosis, addressing the challenges of misdiagnosis and delayed treatment in Alzheimer's patients.

Applying CNN-SVM for Early Detection of Alzheimer's Disease

Ben Rabeh et al. (2023) propose a hybrid approach integrating Convolutional Neural Networks and Support Vector Machines for early Alzheimer's disease prediction. The study leverages MRI scans and transfer learning techniques to enhance classification accuracy. Using the OASIS database, which includes 420 MRI scans (210 normal and 210 with Mild Cognitive Impairment), the model achieves an accuracy of 94.44%. By combining CNN for feature extraction and SVM for classification, the approach outperforms traditional deep learning models like AlexNet and GoogleNet. This hybrid model offers a promising solution for automated early AD detection, improving diagnostic reliability and reducing manual assessment dependency.

A Web-Based Interface for Comprehensive Staging Classification of Alzheimer's Disease Diagnosis Through Convolutional Neural Networks.

Vijaya Kumar Gurralla et al., (2024) propose a web-based system for diagnosing and classifying the stages of Alzheimer's disease using Convolutional Neural Networks (CNN). The approach involves processing MRI images, training a CNN model to classify Alzheimer's into stages early, mild, and moderate—and integrating the model into a user-friendly Python-based graphical interface. The interface provides users with detailed

information on symptoms and precautionary measures for each stage. The model achieves high accuracy and aims to enhance early diagnosis, which is crucial for managing Alzheimer's disease progression.

Alzheimer's Dementia Detection: An Optimized Approach Using ITD of EEG Signals

Sena Yagmur Sen et al. (2024) propose an optimized approach for early-stage Alzheimer's dementia detection using EEG signals. The method utilizes ITD and STFT to convert EEG signals into time-frequency images, which are then analyzed using a convolutional neural network . Experimental results demonstrate that the proposed method outperforms traditional raw EEG signal processing, achieving high accuracy and robustness. The approach provides an efficient, non-invasive technique for early Alzheimer's detection, enhancing diagnostic precision.

CHAPTER 3

ALZHEIMER'S DISEASE THROUGH ANALYSIS OF MRI IMAGES USING DEEP LEARNING TECHNIQUES

Integrates artificial intelligence (AI) with medical imaging to improve the early detection of Alzheimer's disease (AD). This model focuses on deep learning techniques, particularly Convolutional Neural Networks (CNNs), to analyze MRI scans and accurately classify brain structures. By leveraging automated segmentation and classification methods, it enhances precision, reduces human errors, and speeds up diagnosis, which is crucial for early intervention and treatment planning. MRI-based analysis helps in identifying changes in brain anatomy associated with AD progression, making it a valuable tool for clinical applications.

Drawbacks:

- **Data Limitations:** The accuracy of deep learning models heavily depends on the quality and diversity of MRI datasets. A lack of varied samples may affect generalizability across different populations.
- **High Computational Demand:** Deep learning-based image processing requires powerful GPUs and extensive memory, increasing costs and limiting accessibility in resource-constrained healthcare settings.
- **Segmentation Challenges:** The complexity of brain structures and variations in MRI quality can lead to difficulties in precise segmentation, impacting classification accuracy.

- **Model Interpretability:** Although CNNs provide high accuracy, understanding the decision-making process remains a challenge, reducing transparency in medical diagnoses.
- **Real-world Implementation:** Integrating AI-based AD diagnosis into existing healthcare infrastructure may face adoption challenges, requiring significant technological adaptation and regulatory approvals.

CHAPTER 4

LEVERAGING CNN AND SVM FOR EFFECTIVE ALZHEIMER'S DISEASE CLASSIFICATION

The proposed system employs a hybrid approach combining Convolutional Neural Networks and Support Vector Machines to classify Alzheimer's disease stages based on MRI scans. By integrating deep learning and machine learning techniques, this system enhances diagnostic accuracy and reliability.

MRI images undergo several processing stages, starting with preprocessing, feature extraction using CNN, and classification using SVM. CNN automatically extracts spatial and hierarchical features from MRI scans, enabling the system to learn patterns associated with different AD stages: Mild, Very Mild, Moderate, and Demented. These extracted features are then fed into an SVM classifier, which differentiates between these categories based on learned patterns.

To enhance interpretability, Grad-CAM heatmaps highlight the most significant brain regions influencing classification decisions. The hybrid deep learning approach provides an automated, explainable, and robust solution for early Alzheimer's detection.

4.1 DATA PREPROCESSING

Data preprocessing is a critical step in ensuring the consistency, accuracy, and efficiency of the deep learning model.

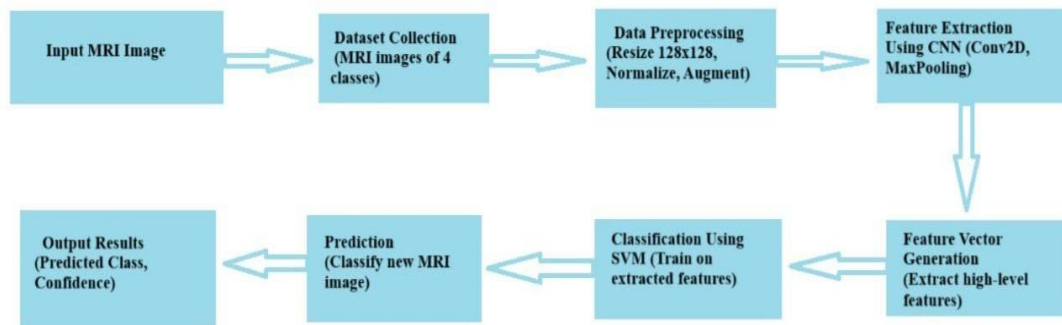


Fig 4.1 Step-by-Step Brain Image Classification

- **Image Resizing and Normalization**

MRI scans come in different sizes and resolutions, which can create inconsistencies in model training. To standardize the input, all images are resized to a uniform dimension. This ensures that the CNN model processes images of the same size, making computations more efficient.

Normalization is another essential step, where pixel intensity values are adjusted to fall within a specific range. This helps the model learn patterns more effectively by reducing variations in image brightness and contrast.

- **Data Augmentation**

Data augmentation artificially increases the dataset size by applying transformations such as rotation, flipping, and contrast adjustments to existing MRI images. This improves model generalization, ensuring that it performs well on unseen data.

For example:

1. Rotation helps the model recognize images from different orientations.
2. Flipping ensures robustness against mirrored scans.

3. Contrast enhancement improves visibility of important brain structures.

- **Label Encoding and One-Hot Encoding**

Since the classification labels (e.g., Mild, Very Mild, Moderate, Demented) are in text format, they must be converted into numerical values for processing. This transformation is done using label encoding, where each class is assigned a unique number.

To further assist the model in understanding class distinctions, one-hot encoding is applied. This technique represents each class as a separate vector, preventing the model from assuming an unintended numerical relationship between categories.

- **Feature Extraction Using CNN**

CNN plays a vital role in automatically learning patterns and structural differences in MRI scans. The convolutional layers in CNN scan images for features such as edges, textures, and shapes that indicate abnormalities in brain structure.

Each layer in the CNN captures progressively complex details:

1. Early layers detect basic features like edges and textures.
2. Deeper layers identify high-level patterns associated with Alzheimer's disease.

By stacking multiple layers, CNN effectively learns hierarchical representations, making it easier to classify different stages of AD.

- **Feature Normalization Using Standard Scaler**

The features extracted by CNN are numerical values that may have different ranges. Feature normalization ensures that all features

contribute equally to the final classification. This is particularly important for SVM, as it relies on well-distributed input data to create optimal decision boundaries between classes.

- **Data Splitting (Train-Test Split)**

To evaluate model performance, the dataset is divided into two parts:

1. Training Set: Used to train the CNN-SVM model.
2. Testing Set: Used to assess the model's accuracy on new data.

4.2 CONVOLUTIONAL NEURAL NETWORK (CNN)

CNN is a deep learning model designed for image analysis, particularly effective in medical imaging tasks like MRI classification. It processes images by applying multiple filters that extract meaningful patterns, such as edges, textures, and structural abnormalities. Through a hierarchical approach, CNN identifies both low-level features (edges, contrast variations) and high-level patterns (brain tissue changes associated with Alzheimer's disease).

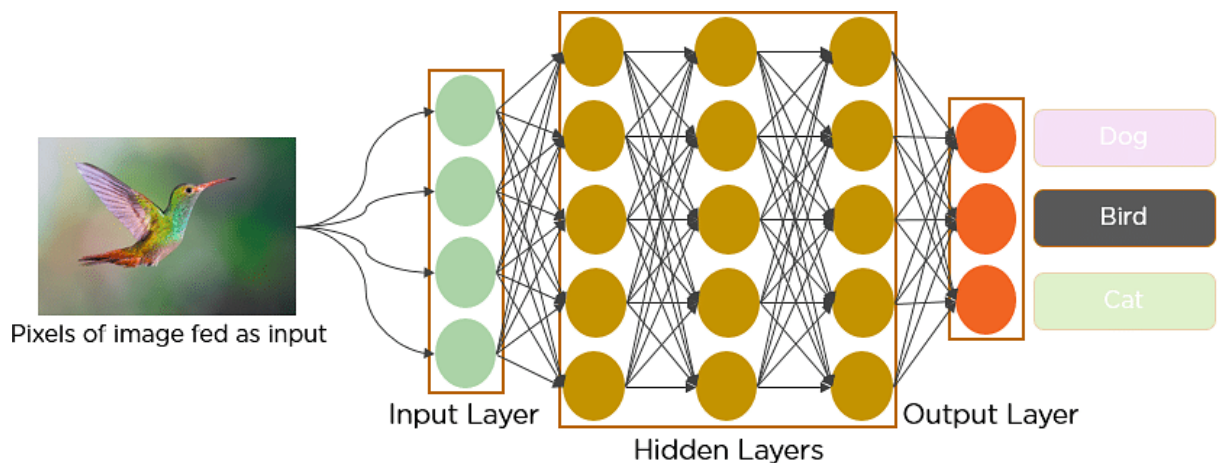


Fig 4.2 CNN to identify the image of a bird

4.3 WORKING OF CNN LAYERS

Convolutional Layer (conv1, conv2)

The convolutional layers extract key features from MRI scans using small filters that slide over the image. The first convolutional layer (conv1) captures basic edges and textures with 32 filters, while the second convolutional layer (conv2) detects complex structures and abnormalities with 64 filters. These layers help identify essential patterns related to Alzheimer's disease.

Activation Function (ReLU)

The ReLU activation function introduces non-linearity by converting negative values to zero while retaining positive values. This ensures that only important features are passed forward, helping the model focus on meaningful patterns in the MRI scan.

Pooling Layer (Max Pooling)

Max pooling reduces the size of feature maps while preserving critical information. It selects the maximum value in a small region, making the model more efficient and less sensitive to small variations in MRI scans. This speeds up processing while retaining key details.

Flatten Layer

After feature extraction, the flatten layer converts the multi-dimensional feature maps into a single long vector. This transformation allows the data to be fed into the fully connected layers for classification.

Fully Connected Layers (fc1, fc2)

The first fully connected layer (fc1) contains 128 neurons and processes the extracted features to recognize patterns. The final fully connected layer (fc2) determines the output, classifying the MRI scan into one of the four Alzheimer's stages: Mild, Very Mild, Moderate, or Demented.

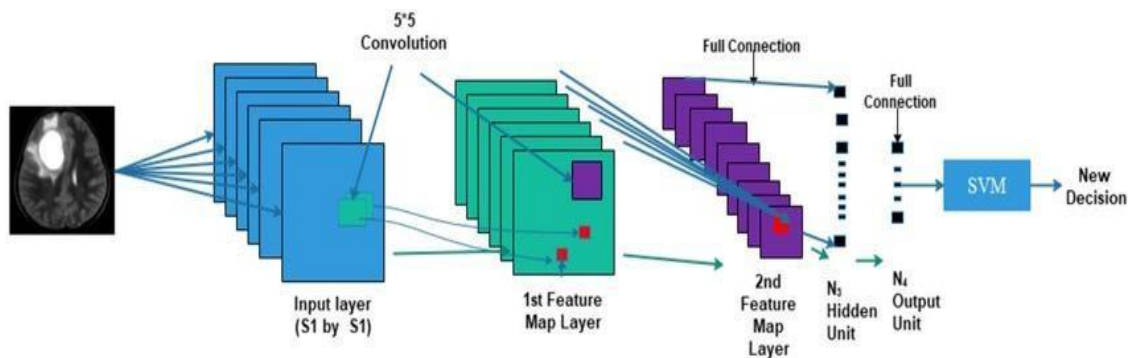


Fig 4.3 Hybrid CNN-SVM Architecture for Alzheimer's Disease

4.4 SUPPORT VECTOR MACHINE (SVM) – CLASSIFICATION

SVM is a machine learning algorithm used to classify the features extracted by the Convolutional Neural Network (CNN) into different Alzheimer's disease stages: Mild, Very Mild, Moderate, and Demented. After CNN extracts meaningful patterns from MRI images, SVM takes these features and finds the optimal hyperplane that best separates the different stages of Alzheimer's disease. It works by maximizing the margin between classes in the feature space, ensuring that new MRI scans can be accurately classified based on their features. The SVM model thus enhances the overall classification accuracy by refining the output from the CNN.

4.5 WORKING OF SVM

SVM as a Classifier in CNN-SVM Model

The Support Vector Machine (SVM) is used as the final classifier in the CNN-SVM hybrid model for Alzheimer's disease detection. The Convolutional Neural Network (CNN) extracts feature representations from MRI scans, which are then classified by SVM into different disease stages.

Feature Extraction from CNN

The CNN processes MRI images and extracts high-dimensional feature vectors that capture important patterns in brain structures. These features are taken from the fully connected layer before classification.

Feature Normalization

Extracted features are normalized using `StandardScaler` to maintain consistency. This step standardizes the data, ensuring better classification performance by SVM.

SVM Training

The normalized features are used to train an SVM classifier with a linear kernel. The model learns to differentiate between Alzheimer's disease stages by identifying the best decision boundary in the feature space.

Prediction and Confidence Scoring

During prediction, new MRI images undergo feature extraction, normalization, and classification by SVM. The `predict_proba` function provides confidence scores, indicating the probability of each classification.

Model Saving and Deployment

The trained SVM model is saved as `svm\_model.pkl` using joblib, allowing future predictions without retraining. It enables efficient and scalable Alzheimer's disease classification.

Role of SVM in CNN-SVM Model

The SVM refines CNN-extracted features, ensuring accurate and interpretable classification. By combining deep feature extraction with robust machine learning classification, the model provides a highly effective system for early Alzheimer's detection.

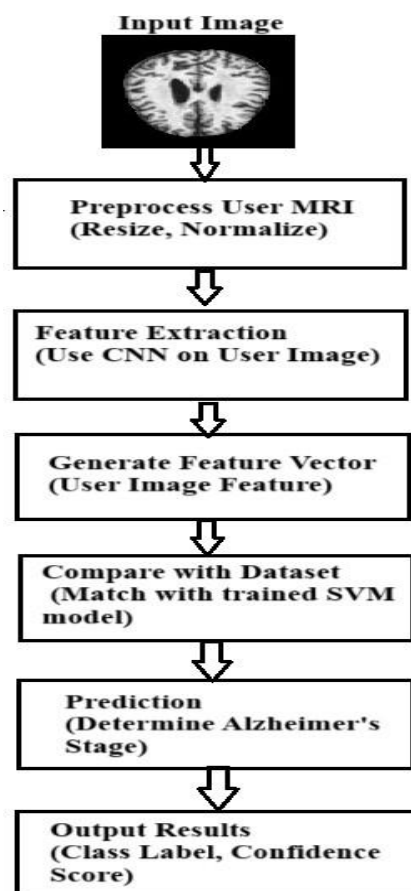


Fig 4.4 Alzheimer's Disease Classification

CHAPTER 5

SYSTEM SPECIFICATION

5.1 HARDWARE REQUIREMENTS

CPU type	:	Intel core i5 processor
Clock speed	:	3.0 GHz
RAM size	:	8 GB
Hard disk capacity	:	500 GB
Keyboard type	:	Internet Keyboard
CD -drive type	:	52xmax

5.2 SOFTWARE REQUIREMENTS

Operating System	:	Windows OS
Programming Language	:	Python(Version3.x)
IDE	:	Visual Studio Code
Database Management System	:	SQLite
Frameworks	:	PyTorch,OpenCV, Streamlit

5.3 ANACONDA (PYTHON DISTRIBUTION)

Anaconda is the ideal environment for running the Alzheimer's MRI Detection application as it supports the dependencies for machine learning and deep learning models.

OVERVIEW

The Anaconda Distribution simplifies the management of the project's dependencies, ensuring compatibility between different libraries required for machine learning tasks like PyTorch, scikit-learn, and OpenCV. By leveraging conda, we ensure that the required libraries for the CNN and SVM models do not conflict with each other.

To work with this project, Anaconda should be installed to manage Python and its environment for smooth running, package management, and ensuring dependencies like torch, torchvision, and scikit-learn are handled.

ANACONDA NAVIGATOR

Anaconda Navigator will be used for managing the environment where the CNN and SVM models will be trained and run. It also provides a GUI for interacting with JupyterLab and Spyder for any data manipulation or debugging tasks.

ANACONDA CLOUD

The Anaconda Cloud provides an easy way to distribute the SVM and CNN models to multiple users while ensuring that the necessary dependencies for each platform (Windows, Linux, macOS) are compatible. If necessary, users can upload custom-built models or datasets to Anaconda Cloud for easy retrieval.

5.4 VISUAL STUDIO CODE

An overview of how to set up VS Code for machine learning projects, including essential tools, extensions, and best practices. VS Code is an excellent choice for machine learning due to its flexibility, ease of use, and

robust extension ecosystem. It explains how to install, run, and use Jupyter Notebooks for data science, including tips, best practices, and examples.

As a web application in which you can create and share documents that contain live code, equations, visualizations as well as text, the Jupyter Notebook is one of the ideal tools to help you to gain the data science skills.

SETTING UP VS CODE FOR MACHINE LEARNING

To begin working on machine learning projects in VS Code, the following setup steps are necessary:

- **Install VS Code:** Download and install the latest version of Visual Studio Code.
- **Install Python:** Ensure that Python is installed on the system. The latest version can be downloaded from Python's official website

Install Essential Extensions:

Python: This extension provides rich support for Python development, including IntelliSense, linting, and debugging.

- **Pylance:** Enhances performance and IntelliSense features for Python coding.
- **Jupyter:** Useful for running and interacting with Jupyter notebooks within VS Code.
- **GitLens:** Helps in managing version control through Git integration.
- **Docker:** Supports containerized machine learning models.
- **Python Docstring Generator:** Assists in generating docstrings for machine learning functions.

VS CODE EXTENSIONS FOR ML

Several extensions within VS Code can significantly enhance the machine learning workflow:

- **Python Extension:** Offers code completion, syntax highlighting, and support for running Python scripts.
- **TensorFlow Snippets:** A collection of code snippets for rapid TensorFlow development.
- **PyTorch Snippets:** Code snippets to aid in the development of PyTorch-based models.
- **scikit-learn Snippets:** Provides snippets to streamline the creation of scikit-learn models and pipelines.
- **Visual Studio IntelliCode:** Uses AI-assisted code suggestions to speed up development.

DEBUGGING MACHINE LEARNING CODE IN VS CODE

VS Code provides a powerful debugging environment for machine learning code. Key features include.

- **Setting Breakpoints:** Breakpoints can be set by clicking next to the line number in the editor's gutter.
- **Running the Debugger:** The debugger can be launched by pressing F5 or using the Run button.
- **Inspecting Variables:** The Variables panel and hovering over variables allows inspection of their current values during debugging.
- **Evaluating Expressions:** The Debug Console allows evaluating expressions and testing parts of the code during the debugging process.

ORGANIZING MACHINE LEARNING PROJECTS IN VS CODE

Efficient organization of machine learning code is crucial for scalability and maintainability. The following directory structure is recommended for machine learning projects:

```
Alzheimers_Detection/
├── dataset/                # MRI image dataset
│   ├── train/             # Training set (Mild, Very Mild, Moderate, Demented)
│   ├── val/               # Validation set
│   ├── test/              # Test set
│   └── images/            # Raw MRI images
├── models/                # Trained models
│   ├── cnn_model.pth      # Trained CNN model
│   └── svm_model.pkl      # Trained SVM model
├── src/                   # Source code
│   ├── train.py           # Script for training the CNN and SVM models
│   ├── user.py            # Script for prediction and inference using the models
│   ├── gradcam.py         # Grad-CAM implementation for explainability
│   ├── retrain.py         # Script for retraining models
│   ├── preprocessing.py   # Data preprocessing functions (e.g., resizing, augmentat
├── app.py                 # Streamlit web app for interfacing with the models
└── requirements.txt       # Python dependencies
```

Fig 5.1 Directory Structure of the Alzheimer's Detection System

RUNNING AND TESTING IN ML MODELS

VS Code facilitates running and testing machine learning models with the following capabilities:

- **Running Scripts:** Python scripts can be run directly from the editor by pressing Ctrl + F5 (run without debugging) or F5 (run with debugging).
- **Unit Testing:** VS Code integrates with testing frameworks such as unittest, pytest, and nose, enabling automated testing of machine learning model

BEST PRACTICES FOR MACHINE LEARNING IN VS CODE

To manage machine learning projects effectively within VS Code, the following best practices are recommended:

- **Version Control:** Utilize Git for version control to track changes and facilitate collaboration. VS Code's Git integration makes version management seamless.
- **Code Documentation:** Proper documentation through comments and docstrings is essential for complex machine learning models. This ensures the readability and maintainability of code.
- **Modular Code:** Structuring code into modular functions or classes—such as separating data preprocessing, model training, and evaluation—helps in maintaining clean and scalable code.
- **Experiment Tracking:** Tools like MLflow or TensorBoard can be used to track experiments, visualize training progress, and log hyperparameters and metrics.

5.5 PYTHON

BACKEND DEVELOPMENT: Python can be used for server-side development, implementing the backend logic of the code review tool. Frameworks like Django or Flask can facilitate the development of robust and scalable web applications.

MACHINE LEARNING INTEGRATION: Python is widely used in the machine learning ecosystem. Libraries like TensorFlow, PyTorch, or scikit-learn can be employed to implement machine learning algorithms for code analysis, identifying patterns, and providing automated feedback.

SCRIPTING AND AUTOMATION: Python's versatility makes it suitable for scripting tasks and automating various aspects of the code review process. This includes tasks such as data processing, integration with external systems, and managing workflows.

API DEVELOPMENT: Python can be used to develop APIs (Application Programming Interfaces) for communication between different components of the system, enabling seamless interactions between the frontend and backend.

5.6 WINDOWS

Windows is a series of operating systems developed by Microsoft. Each version of Windows includes a graphical user interface, with a desktop that allows users to view files and folders in windows. For the past two decades, Windows has been the most widely used operating system for personal computers PCs.

CHAPTER 6

IMPLEMENTATION AND RESULT

6.1 INTRODUCTION

The Alzheimer's disease detection system utilizes a hybrid model of Convolutional Neural Networks (CNN) and Support Vector Machines (SVM) to classify MRI scans into four stages: Mild, Very Mild, Moderate, and Demented. The system first preprocesses the MRI images, which are then fed into a CNN for feature extraction. These features are subsequently passed through an SVM classifier for final prediction. The model's performance is enhanced with Grad-CAM for generating heatmaps, which help visualize the regions of the MRI scan that contributed to the decision, providing an explainable aspect of the model's predictions. The system also features real-time retraining, allowing it to be updated with new data to improve its accuracy over time. The web application, built with Streamlit, enables users to upload MRI scans, view prediction results, and explore prediction history, offering an interactive platform for Alzheimer's disease detection.

6.2 CODING

train.py

```
import os
```

```
import torch
```

```

import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import numpy as np
import joblib
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, Subset

# Define CNN Model
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()

        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()

        self.fc1 = nn.Linear(64 * 32 * 32, 128)
        self.fc2 = nn.Linear(128, 4) # 4 output classes
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

```

```
# Dataset Preparation
```

```
transform = transforms.Compose([ transforms.Resize((128, 128)),
transforms.ToTensor()])
```

```
dataset_path = "D:\\PROJECT\\Alzheimer\\dataset"
```

```
dataset = datasets.ImageFolder(root=dataset_path, transform=transform)
```

```
# Apply Label Encoding for class labels
```

```
label_encoder = LabelEncoder()
```

```
encoded_labels = label_encoder.fit_transform([dataset.classes[i] for i in
dataset.targets])
```

```
# Split dataset
```

```
train_indices, test_indices = train_test_split(np.arange(len(dataset)),
```

```
test_size=0.2, stratify=encoded_labels, random_state=42)
```

```
train_dataset = Subset(dataset, train_indices) test_dataset = Subset(dataset,
test_indices)
```

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True) test_loader =
DataLoader(test_dataset, batch_size=32, shuffle=False) print("Dataset uploaded
properly")
```

```
# Train CNN model
```

```
model = CNNModel()
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
print("Training CNN model...")
```

```

def train_cnn():
    model.train()
    for epoch in range(10):
        for images, labels in train_loader:
            images = images.to(torch.float32) # Ensure correct
            dtype labels = labels.to(torch.long)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
        print(f"Epoch {epoch+1}, Loss: {loss.item()}")
    torch.save(model.state_dict(), "models/cnn_model.pth")

```

Extract CNN Features & Train SVM

```

def extract_features():
    model.eval()
    features = []
    labels = []
    with torch.no_grad():
        for images, lbls in train_loader:
            images = images.to(torch.float32)
            outputs = model(images)
            features.extend(outputs.numpy())
            labels.extend(lbls.numpy())
    return np.array(features), np.array(labels)

def train_svm():
    features, labels = extract_features()

```

```
# Normalize features
```

```
scaler = StandardScaler()
```

```
    features = scaler.fit_transform(features)
```

```
    svm = SVC(kernel='linear', probability=True)
```

```
    svm.fit(features, labels)
```

```
# Save models & encoders
```

```
joblib.dump(svm, "models/svm_model.pkl")
```

```
joblib.dump(scaler, "models/scaler.pkl")
```

```
joblib.dump(label_encoder, "models/label_encoder.pkl")
```

```
if __name__ == "__main__":
```

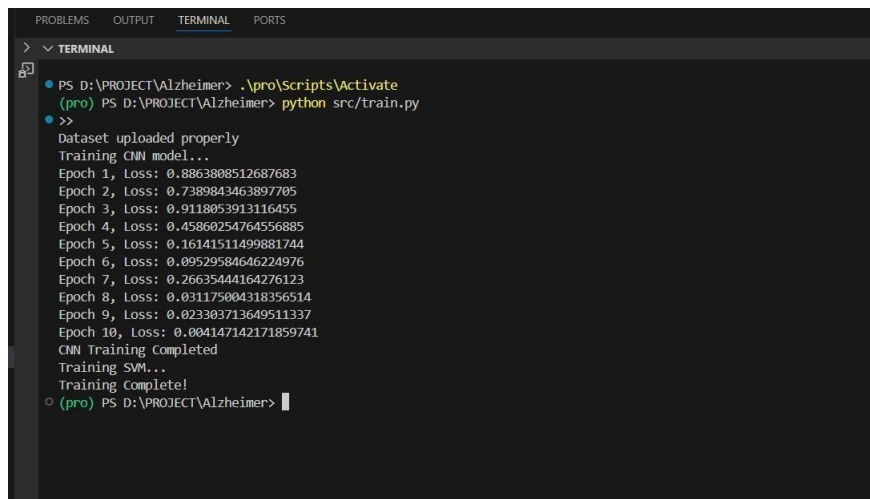
```
    train_cnn()
```

```
    print("CNN Training Completed")
```

```
    print("Training SVM...")
```

```
    train_svm()
```

```
    print("Training Complete!")
```



```

> ▾ TERMINAL
● PS D:\PROJECT\Alzheimer> .\pro\Scripts\Activate
(pro) PS D:\PROJECT\Alzheimer> python src/train.py
● >>
Dataset uploaded properly
Training CNN model...
Epoch 1, Loss: 0.8863808512687683
Epoch 2, Loss: 0.7389843463897705
Epoch 3, Loss: 0.9118053913116455
Epoch 4, Loss: 0.45860254764556885
Epoch 5, Loss: 0.16141511499881744
Epoch 6, Loss: 0.09529584646224976
Epoch 7, Loss: 0.26635444164276123
Epoch 8, Loss: 0.031175004318356514
Epoch 9, Loss: 0.023303713649511337
Epoch 10, Loss: 0.004147142171859741
CNN Training Completed
Training SVM...
Training Complete!
○ (pro) PS D:\PROJECT\Alzheimer>

```

Fig 6.1 Training Process of CNN and SVM for Alzheimer's Detection

model.py

```

import torch

import torch.nn as nn

import torch.nn.functional as F

# Define CNN Model
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()

        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

        self.flatten = nn.Flatten()

        self.fc1 = nn.Linear(64 * 32 * 32, 128)

        self.fc2 = nn.Linear(128, 4) # 4 classes (Mild, Very Mild, Moderate,
Demented)

        self.relu = nn.ReLU()

    def forward(self, x):

        x = self.pool(F.relu(self.conv1(x)))

        x = self.pool(F.relu(self.conv2(x)))

        x = self.flatten(x)

        x = F.relu(self.fc1(x))

        x = self.fc2(x)

return

```

database.py

```

import sqlite3

# Connect to SQLite database (creates file if not exists)
conn = sqlite3.connect("patient_history.db")
cursor = conn.cursor()

# Create table if it doesn't exist
cursor.execute("""
    CREATE TABLE IF NOT EXISTS predictions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        image_path TEXT,
        prediction TEXT,
        confidence REAL )""")

conn.commit()

conn.close()

```

gradcam.py

```

import torch

import torch.nn.functional as F

import numpy as np

import cv2

class GradCAM:

```

```

def _init_(self, model, target_layer):
    self.model = model
    self.target_layer = self.find_layer(target_layer)
    self.gradients = None
    self.activations = None
    self.hook_layers()

def find_layer(self, layer_name):
    layers = dict(self.model.named_modules())
    return layers[layer_name]

def hook_layers(self):
    def forward_hook(module, input, output):
        self.activations = output.detach()

    def backward_hook(module, grad_input, grad_output):
        self.gradients = grad_output[0].detach()
        self.target_layer.register_forward_hook(forward_hook)
        self.target_layer.register_backward_hook(backward_hook)

def generate(self, image_tensor, class_idx):
    self.model.zero_grad()
    output = self.model(image_tensor)
    class_score = output[:, class_idx]
    class_score.backward()
    weights = self.gradients.mean(dim=[2, 3], keepdim=True)
    cam = torch.sum(weights * self.activations, dim=1)

```

```

    cam = F.relu(cam)

    cam = cam.squeeze().cpu().numpy()

    cam = cv2.resize(cam, (224, 224))

    cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)  #
    Normalize

    return cam

```

user.py

```

import os

import torch

import torchvision.transforms as transforms

from PIL import Image

import numpy as np

import joblib

import torch.nn as nn

import cv2

import sqlite3

# Load trained models
cnn_model_path = "models/cnn_model.pth"
svm_model_path = "models/svm_model.pkl"
scaler_path = "models/scaler.pkl"
label_encoder_path = "models/label_encoder.pkl"

```

```
# Define CNN Model
```

```
class CNNModel(nn.Module):
```

```
    def _init_(self):
```

```
        super(CNNModel, self)._init_()
```

```
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
```

```
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.flatten = nn.Flatten()
```

```
        self.fc1 = nn.Linear(64 * 32 * 32, 128)
```

```
        self.fc2 = nn.Linear(128, 4)
```

```
        self.relu = nn.ReLU()
```

```
    def forward(self, x):
```

```
        x = self.pool(self.relu(self.conv1(x)))
```

```
        x = self.pool(self.relu(self.conv2(x)))
```

```
        x = self.flatten(x)
```

```
        x = self.relu(self.fc1(x))
```

```
        x = self.fc2(x)
```

```
        return x
```

```
# Load models
```

```
cnn_model = CNNModel()
```

```
cnn_model.load_state_dict(torch.load(cnn_model_path,
map_location=torch.device("cpu")))
```

```

cnn_model.eval()

svm_model = joblib.load(svm_model_path)
scaler = joblib.load(scaler_path)
label_encoder = joblib.load(label_encoder_path)


# SQLite database setup
DB_PATH = "patient_history.db"

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS history
                (id INTEGER PRIMARY KEY AUTOINCREMENT,
                 image_path TEXT, prediction TEXT, confidence REAL)""")
    conn.commit()
    conn.close()

init_db()

def save_to_db(image_path, prediction, confidence):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("INSERT INTO history (image_path, prediction, confidence)
VALUES (?, ?, ?)",
              (image_path, prediction, confidence))
    conn.commit()

```

```
conn.close()

# Image Preprocessing
def preprocess_image(image_path):
    image = Image.open(image_path).convert("RGB")
    transform = transforms.Compose([
        transforms.Resize((128, 128)),
        transforms.ToTensor()])
    return transform(image).unsqueeze(0)

# Prediction Function
def predict(image_path):
    image_tensor = preprocess_image(image_path)

# Extract CNN feature vector
    with torch.no_grad():
        cnn_features = cnn_model(image_tensor).numpy()

# Normalize features before SVM prediction
    cnn_features = scaler.transform(cnn_features)

# Predict with SVM
    prediction = svm_model.predict(cnn_features)[0]
```

```

confidence = np.max(svm_model.predict_proba(cnn_features))
predicted_label = label_encoder.inverse_transform([prediction])[0]

# Save to database
save_to_db(image_path, predicted_label, confidence)

return predicted_label, confidence

# Grad-CAM Heatmap Function
def generate_heatmap(image_path):
    image_tensor = preprocess_image(image_path)
    image_tensor.requires_grad_()

    # Forward pass
    output = cnn_model(image_tensor)
    class_idx = output.argmax().item()

    # Backpropagation
    cnn_model.zero_grad()
    output[0, class_idx].backward()

    # Extract gradients from the last convolutional layer
    gradients = cnn_model.conv2.weight.grad.mean(dim=[0, 2, 3]).detach().numpy()

```



```

# Convert image tensor to numpy for visualization

image_np = image_tensor.detach().squeeze().permute(1, 2, 0).numpy()


# Normalize and apply heatmap

heatmap = np.maximum(gradients, 0)

heatmap = cv2.resize(heatmap, (128, 128))

if heatmap.max() > heatmap.min():

    heatmap = (heatmap - heatmap.min()) / (heatmap.max() -
heatmap.min())

    heatmap = cv2.applyColorMap(np.uint8(255 * heatmap),
cv2.COLORMAP_JET)


# Overlay heatmap on the image

overlay = cv2.addWeighted(cv2.cvtColor(image_np, cv2.COLOR_RGB2BGR), 0.6, heatmap, 0.4, 0)


# Save and return heatmap

heatmap_path = "heatmap.jpg"

cv2.imwrite(heatmap_path, overlay)

return heatmap_path


# Retrain Model Function

def retrain_model()

    os.system("python src/train.py")

```

app.py

```
import streamlit as st
```

```
import os
```

```
import time
```

```
from src.user import predict, generate_heatmap, retrain_model, DB_PATH
```

```
import sqlite3
```

```
import pandas as pd
```

```
# Set Page Config
```

```
st.set_page_config(page_title="Alzheimer's MRI Detector", page_icon=" ",
layout="wide")
```

```
# Sidebar Navigation
```

```
st.sidebar.title("Navigation")
```

```
page = st.sidebar.radio("Go to", ["Introduction", "Predict", "History",
"Retrain Model"])
```

```
# Theme Toggle
```

```
dark_mode = st.sidebar.checkbox("Dark Mode")
```

```
# Apply custom dark mode style
```

```
if dark_mode:
```

```
    st.markdown("""
```

```

<style>

    body { background-color: #121212; color: white; }

    .stApp { background-color: #121212; }

</style>

""", unsafe_allow_html=True)

# Ensure necessary directories exist
if not os.path.exists("uploads"):
    os.makedirs("uploads")

# Introduction Page
if page == "Introduction":
    st.title("Alzheimer's Disease Detection Using MRI")
    st.markdown("""
### About This Project

    Alzheimer's disease is a progressive neurological disorder that affects
    memory and cognitive function. Early detection is crucial for managing and
    slowing its progression.

    This application uses Convolutional Neural Networks (CNN) and Support
    Vector Machines (SVM) to analyze MRI scans and classify them into
    different stages of Alzheimer's disease.

    Key Features:

    MRI-based classification of Alzheimer's stages

```

Grad-CAM heatmap visualization for explainability

Real-time model retraining for improved accuracy

Upload an MRI scan in the Predict section to get started.

```
""")
```

```
# Display Sample MRI Image
```

```
sample_img_path = "D:/PROJECT/Alzheimer/dataset/NonDemented/1(10).jpg"
```

```
if os.path.exists(sample_img_path):
```

```
    st.image(sample_img_path, caption="Sample MRI Scan",
use_container_width=300)
```

```
else:
```

```
    st.warning("Sample MRI image not found! Please place an image at
assets/sample_mri.jpg.")
```

```
# Prediction Page
```

```
elif page == "Predict":
```

```
    st.title("MRI Scan Prediction")
```

```
    uploaded_file = st.file_uploader("Upload MRI Image", type=["jpg",
"png", "jpeg"])
```

```
if uploaded_file:
```

```
    image_path = os.path.join("uploads", uploaded_file.name)
```

```
    with open(image_path, "wb") as f:
```

```
        f.write(uploaded_file.getbuffer())
```

```

        st.image(image_path, caption="Uploaded MRI", width=300)
    if st.button("Analyze MRI"):
        with st.spinner("Processing..."):
            prediction, confidence = predict(image_path)
            heatmap_path = generate_heatmap(image_path)
            st.success(f"Prediction: {prediction}")
            st.info(f"Confidence Score: {confidence:.2f}")
        st.image(heatmap_path, caption="Grad-CAM Heatmap", width=300)

# History Page
elif page == "History":
    st.title(" Prediction History")

    # Fetch history from database
    conn = sqlite3.connect(DB_PATH)

    df = pd.read_sql_query("SELECT * FROM history ORDER BY id
DESC", conn)

    conn.close()

    if not df.empty:
        st.dataframe(df)
    else:
        st.warning("No history available.")

```

```
# Retrain Model Page

elif page == "Retrain Model":

    st.title("Retrain Model")

    st.write("Use this feature to retrain the model with new data.")

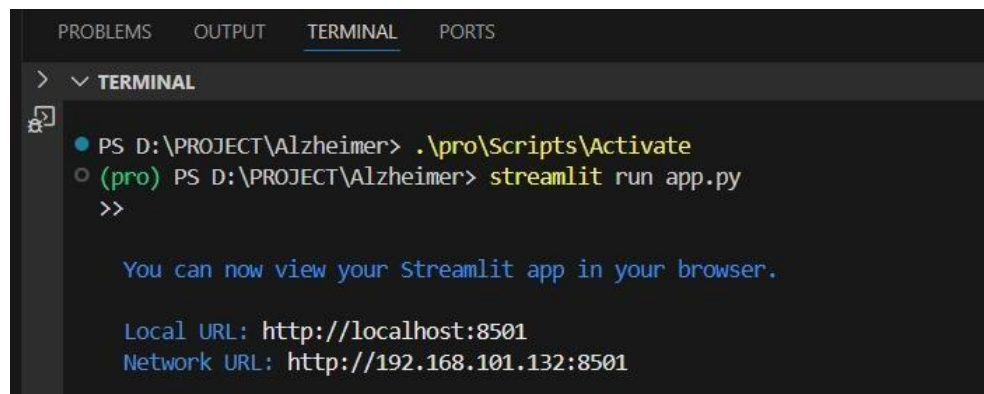
    if st.button("Retrain Now"):

        with st.spinner("Retraining Model... This may take some time."):

            retrain_model()

            time.sleep(5) # Simulate training time

        st.success("Model retrained successfully!")
```



The image shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is selected. Below the tabs, there is a list of commands and their outputs. The first command is 'PS D:\PROJECT\Alzheimer> .\pro\Scripts\Activate', which is followed by a green prompt '(pro)'. The second command is '(pro) PS D:\PROJECT\Alzheimer> streamlit run app.py', followed by a green prompt '>>'. Below the commands, there is a message: 'You can now view your Streamlit app in your browser.' followed by two lines of URLs: 'Local URL: http://localhost:8501' and 'Network URL: http://192.168.101.132:8501'.

```
PROBLEMS  OUTPUT  TERMINAL  PORTS

>  ▾  TERMINAL
● PS D:\PROJECT\Alzheimer> .\pro\Scripts\Activate
○ (pro) PS D:\PROJECT\Alzheimer> streamlit run app.py
  >>

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.101.132:8501
```

Fig 6.2 Training Performance of CNN Model on Alzheimer's Dataset

OUTPUT

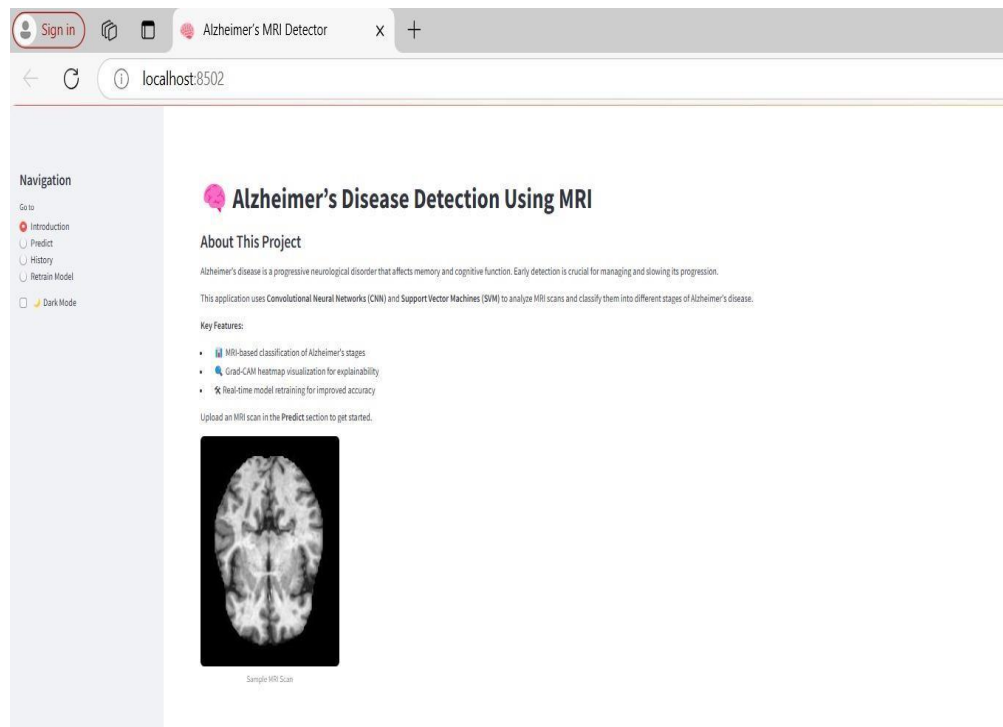


Fig 6.3 Introduction

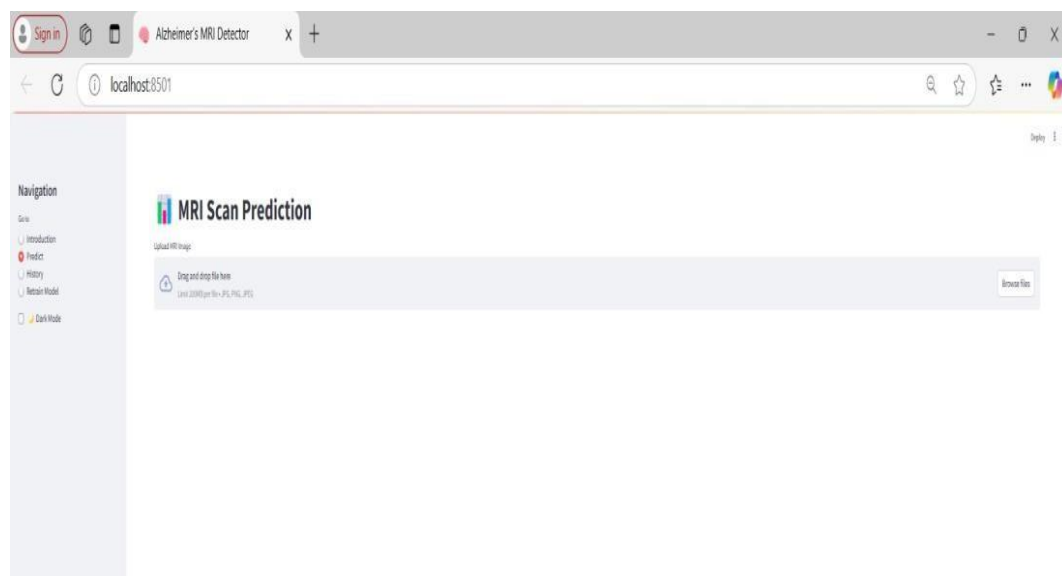


Fig 6.4 Uploading Image

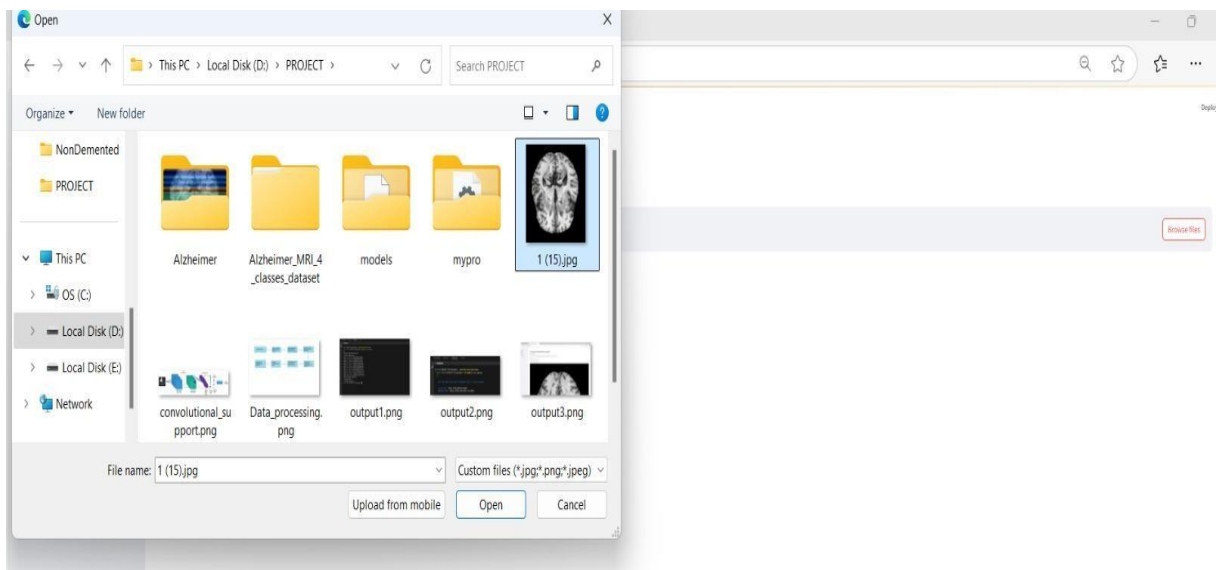


Fig 6.5 Selecting MRI Image

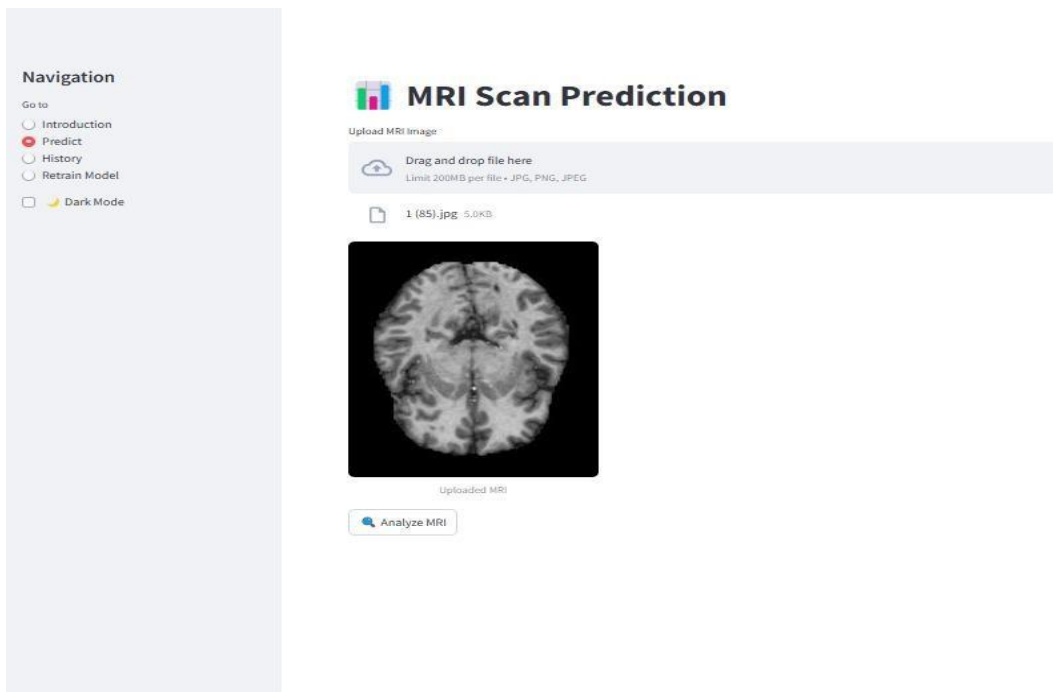


Fig 6.6 Analyze MRI

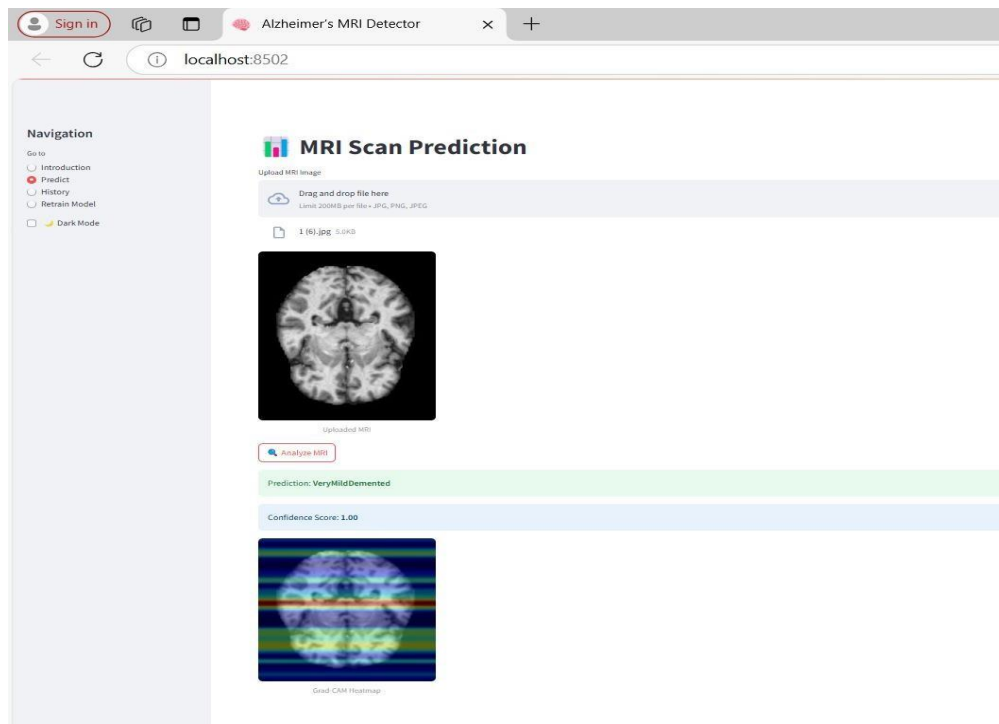


Fig 6.7 Result

Prediction History

	id	image_path	prediction	confidence
0	16	uploads\1 (85).jpg	NonDemented	1
1	15	uploads\2 (23).jpg	NonDemented	1
2	14	uploads\4 (15).jpg	MildDemented	0.9998
3	13	uploads\2 (50).jpg	NonDemented	1
4	12	uploads\1 (45).jpg	NonDemented	0.5744
5	11	uploads\1 (39).jpg	VeryMildDemented	0.9946
6	10	uploads\32 (2).jpg	ModerateDemented	0.8821
7	9	uploads\1 (9).jpg	VeryMildDemented	0.9797
8	8	temp\1 (2).jpg	ModerateDemented	0.8976
9	7	temp\1 (2).jpg	ModerateDemented	0.8976

Fig 6.8 Prediction History

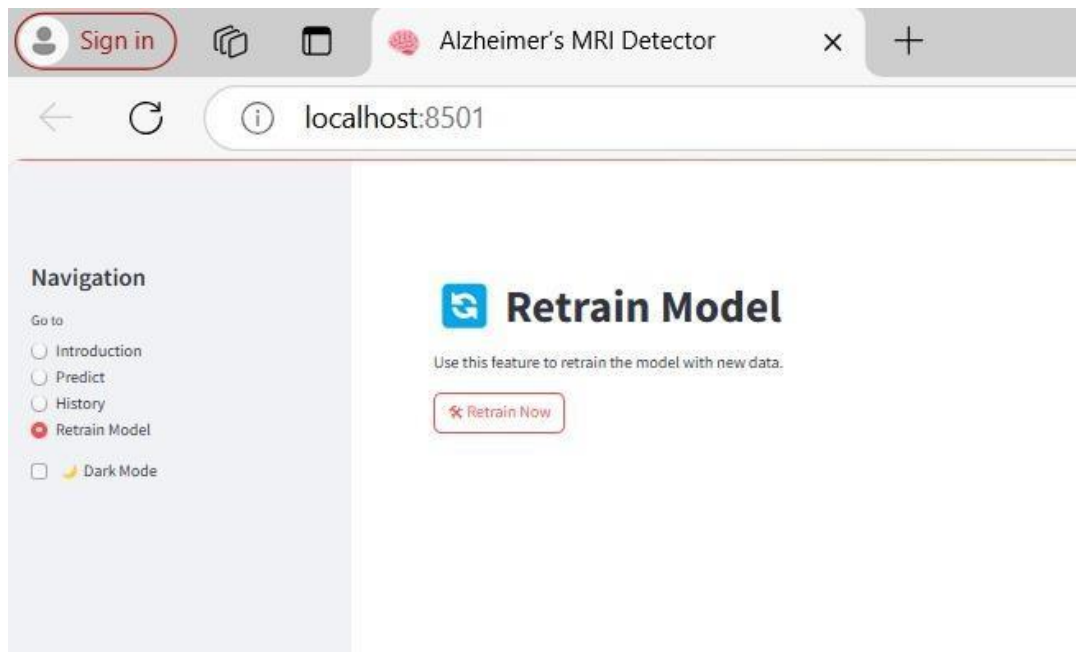


Fig 6.9 Retrain Model

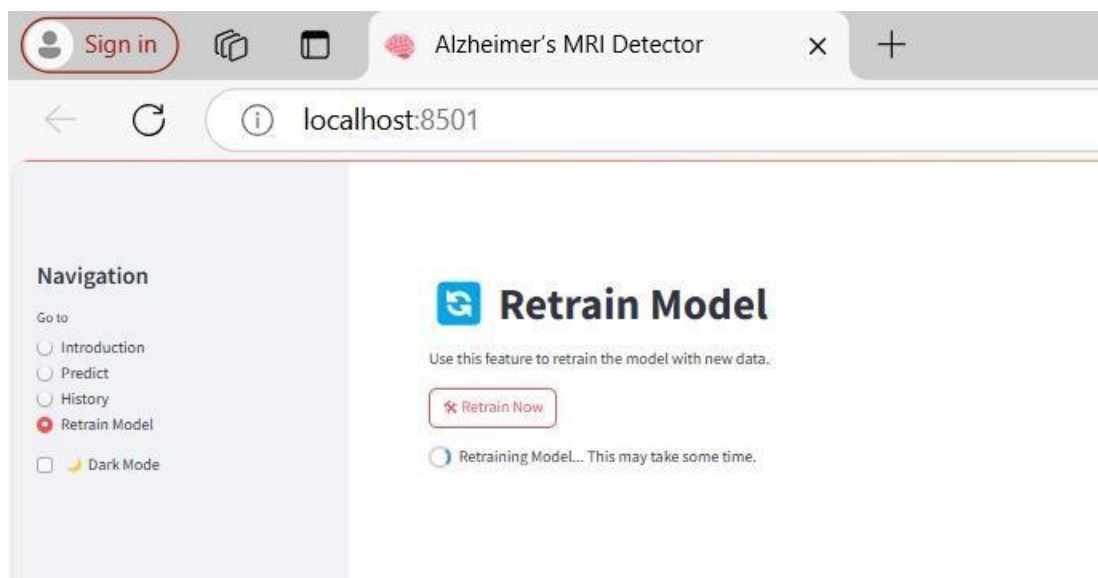


Fig 6.10 Model Training

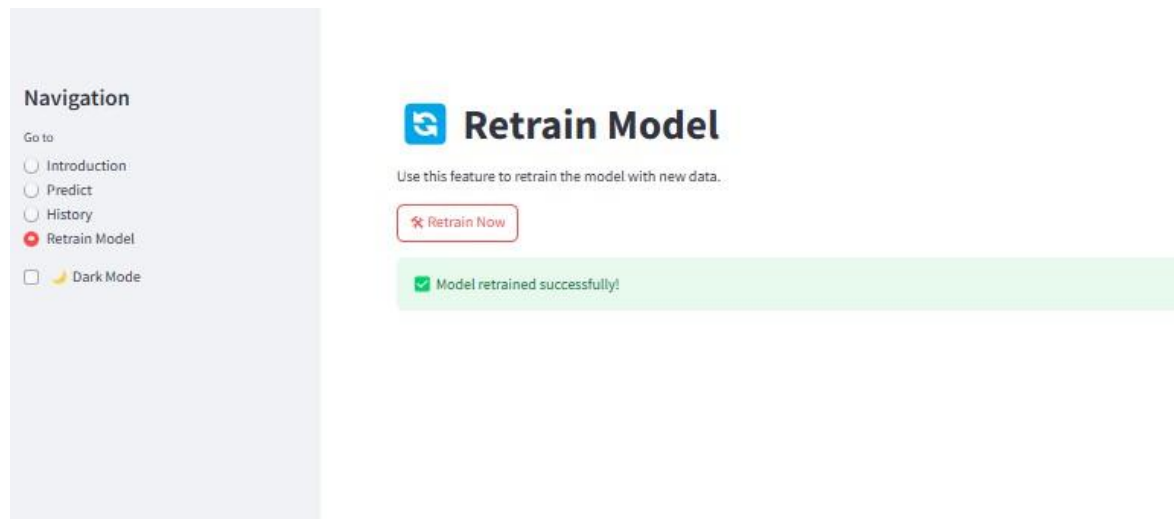


Fig 6.11 Result Of Retrain Model

CHAPTER7

CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION

The hybrid CNN-SVM model for Alzheimer's disease classification has demonstrated high accuracy and reliability in distinguishing different stages of the disease using MRI scans. By leveraging the feature extraction capabilities of CNN and the robust classification power of SVM, the model effectively categorizes MRI images into four classes: Very Mild Demented, Mild Demented, Moderate Demented, and Non-Demented. The integration of a user-friendly interface with real-time classification, patient history storage, and Grad-CAM heatmap visualization enhances the model's applicability in clinical settings. Additionally, the model's retraining functionality ensures continuous improvement with new data, making it a dynamic and adaptive system.

Despite its promising results, further enhancements are needed to improve generalization across diverse datasets, increase interpretability through advanced explainable AI techniques, and explore multi-modal approaches that incorporate additional diagnostic information such as genetic data and cognitive test scores.

By addressing these aspects, the model has the potential to become a powerful decision-support tool in early Alzheimer's detection, aiding healthcare professionals in making timely and accurate diagnoses while continuously evolving to meet real-world medical demands.

FUTURE ENHANCEMENT

While the proposed model achieves high performance, several areas for improvement remain:

1. Expanding Dataset and Improving Generalization

Train on a larger, more diverse dataset from multiple sources to ensure better generalization across different patient demographics. Incorporate different MRI modalities (e.g., PET scans, fMRI) to enhance classification robustness. Address data imbalance issues through advanced augmentation techniques.

2. Enhancing Explainability and Trust in AI Predictions

Improve the Grad-CAM heatmap to better visualize critical brain regions affecting classification. Implement explainable AI (XAI) methods to increase transparency in decision-making. Develop interactive visualization dashboards for healthcare professionals to better understand model predictions.

3. Multi-Modal and Hybrid Approaches

Explore the integration of genetic data, clinical history, and cognitive test scores alongside MRI images. Implement attention-based deep learning models (e.g., Vision Transformers) for improved feature extraction. Incorporate semi-supervised or self-supervised learning to utilize unlabeled MRI scans effectively.

REFERENCES

1. Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
2. Keiron O’shea and Ryan Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
3. Sergey Korolev, Amir Safiullin, Mikhail Belyaev, and Yulia Dodonova. Residual and plain convolutional neural networks for 3d brain mri classification. In *2017 IEEE 14th international symposium on biomedical imaging (ISBI 2017)*, pages 835–838. IEEE, 2017.
4. Martina Zveřová. Clinical aspects of alzheimer’s disease. *’ Clinical biochemistry*, 72:3–6, 2019.
5. Bharat Richhariya, Muhammad Tanveer, Ashraf Haroon Rashid, Alzheimer’s Disease Neuroimaging Initiative, et al. Diagnosis of alzheimer’s disease using universum support vector machine based recursive feature elimination (usvm-rfe). *Biomedical Signal Processing and Control*, 59:101903, 2020.
6. Md Mohsin Kabir, Parisa Mehera, Alope Kumar Saha, MF Mridha, Md Abdul Hamid, and Muhammad Mostafa Monowar. Cnn-nsvm architecture for skin lesion classification using non-dermoscopic digital image. In *2021 Joint 10th International Conference on Informatics, Electronics & Vision (ICIEV) and 2021 5th International Conference on Imaging, Vision & Pattern Recognition (icIVPR)*, pages 1–7. IEEE, 2021.

7. Angona Biswas and Md Saiful Islam. Hybrid cnn-svm model for brain tumor classification utilizing different datasets. In 2021 International Conference on Electronics, Communications and Information Technology (ICECIT), pages 1–4. IEEE, 2021.
8. Mohammad Omid Khairandish, Meenakshi Sharma, Vishal Jain, Jyotir Moy Chatterjee, and NZ Jhanjhi. A hybrid cnn-svm threshold segmentation approach for tumor detection and classification of mri brain images. *Irbm*, 43(4):290–299, 2022.
9. Heejong Kim, Shin Won Kang, Jae-Hun Kim, Himanshu Nagar, Mert Sabuncu, Daniel JA Margolis, and Chan Kyo Kim. The role of ai in prostate mri quality and interpretation: Opportunities and challenges. *European Journal of Radiology*, 165:110887, 2023.
10. Moez Krichen. Convolutional neural networks: A survey. *Computers*, 12(8):151, 2023.
11. Chahd M Chabib, Leontios J Hadjileontiadis, and Aamna Al Shehhi. Deepcurvmri: Deep convolutional curvelet transform-based mri approach for early detection of alzheimer’s disease. *IEEE Access*, 11:44650–44659, 2023.
12. Murat Avcı, ar and Kemal Polat. Classifying alzheimer’s disease based on a convolutional neural network with mri images. *Journal of Artificial Intelligence and Systems*, 5(1):46–57, 2023.
13. N Yuvaraj, T Preethi, AC Sumathi, and KR Sri Preethaa. Alzheimer disease classification based on multimodel deep convolutional neural network using mri images. In *AIP conference proceedings*, volume 2764. AIP Publishing, 2023.
14. Yori Pusparani, Chih-Yang Lin, Yih-Kuen Jan, Fu-Yu Lin, Ben-Yi Liao, Peter Ardianto, Isack Farady, John Sahaya Rani Alex, Jeetashree Aparajeeta, Wen-Hung Chao, and Chi-Wen Lung. Diagnosis of alzheimer’s disease using CNN *IEEE Access*, 11:61688–61697, 2023.

15. Md Masud Rana, Md Manowarul Islam, Md Alamin Talukder, Md Ashraf Uddin, Sunil Aryal, Naif Alotaibi, Salem A Alyami, Khondokar Fida Hasan, and Mohammad Ali Moni. A robust and clinically applicable deep learning model for early detection of alzheimer's. *IET Image Processing*, 17(14):3959–3975, 2023.
16. Ahmed A Abd El-Latif, Samia Allaoua Chelloug, Maali Alabdulhafith, and Mohamed Hammad. Accurate detection of alzheimer's disease using lightweight deep learning model on mri data. *Diagnostics*, 13(7):1216, 2023